Forward Propagation of Information

Teaching The Maths how to speak English: the compiled monad `ptol` and the scaled boxkite Mind's Eye monad

Cody Michael Allison (Michael Rendier), independent researcher · the.wandering.god@gmail.com · ORCID 0009-0007-7239-6760
Written by an AI assistant: Claude Sonnet 5.5 (`claude-sonnet-5-5`), with the author's `cs-paper-code-conventions` skill running (`ContextPlease/claude/skills/cs-paper-code-conventions/SKILL.md`)
Date: 2026-10-16 · **Status:** first full draft · **Code:** `VAPMIP`, commit `596e4a8` · **Repository prefix:** every path below, written `REPO/directory/file`, lives under `https://github.com/michaelrendier/`

Read this first: who wrote this, and how

The assistant. This paper was written by a transformer language model, Claude Sonnet 5.5 (`claude-sonnet-5-5`), working with the author's `cs-paper-code-conventions` skill active; the skill decides how code is shown in a paper. Every number in the text that comes from code was produced by `VAPMIP/paper/evidence.py` when `VAPMIP/paper/make_paper.py` last ran; none is typed in by hand.

The author. The author is a software engineer, not a trained mathematician or linguist, and says so. Every statement that is not attributed to a cited source should be read as unverified until an expert has checked it.

The method. The author does not tell the assistant what result he is hoping for. Tasks are given as measurements ("run it", "chase that to its source"), so that he is not handed the answers he wants. This is a safeguard, not a guarantee; the real safeguards are fresh runs, exact comparison where an exact answer exists, and failures kept in the text.

What kind of paper this is. An engineering white paper about two pieces of finished code, shown running. It contains no proofs and makes no prediction. It describes the mechanism and shows the behaviour; it does not reproduce the Monad's source (§2.3). Every result is a measurement of the code as it stands at the commit above; nothing was changed to obtain it.

Abstract

Language models have become costly in four ways at once. They draw large amounts of power. They are trained by backpropagation, which keeps development on clusters and off the developer's own machine. They spend compute to rediscover the relations between words that WordNet already records. And their weights and training data are closed, so a reader cannot see why a model says what it says, or who decided that it should. This paper describes an engine, the Monad, built to avoid all four, and the attempt to teach a single piece of mathematics, the Null Operator ⌀_RB (typed 0_RB), to speak English. It runs offline on one laptop CPU (Intel Core i7-8550U, 2018, no GPU, no network). It has no gradient descent and no trained weights: a prompt becomes one sixteen-component element by a fixed projection, the engine reads that element along a fixed geometry, and words come from a store whose semantic centre is WordNet's own relations. It exists as two pieces of working code, a compiled monad (`ptol`) and the Python monad it was built from (`scaled_mind_eye_boxkite_kernel_monad.py`), and the paper presents both in the order the code runs, with every figure produced by a script that can be re-run. Energy was measured as the whole laptop's discharge on battery: about

66.6 µWh per reply in the compiled REPL (recorded 2026-10-13), about 85.1 µWh per warm reply in the Python monad, and about 30 mWh for a sentence through the compiled monad's Python bridge. The publicly stated figures for one typical language-model query are 0.24 to 0.34 Wh (Vahdat et al., 2025; Altman, 2025; You, 2025). The energy is the result the design aimed at; the language is not yet there. The paper records where the replies fall short of answering the prompt, and where the two programs, and the code and its own comments, disagree. The Monad's source is not reproduced; the mechanism, tables and transcripts are.

---

1. Introduction

Language models have become costly in four ways at once. They draw large amounts of power to train and to run. They are trained by backpropagation, which keeps development on clusters instead of on the machine in front of the developer. They spend compute to rediscover structure, the relations between words, that language already has and that lexical resources such as WordNet record directly. And their weights and training data are closed, so a reader cannot tell why a model says what it says, or who decided that it should.

This paper describes an engine built to avoid each of the four. We call it the Monad. It is a program, written in C and Python, that runs on one laptop CPU with no network and no GPU. It has no gradient descent and no trained weights: a prompt is turned into a single sixteen-component element by a fixed projection, and words are read from a store whose semantic centre is WordNet's own relations. Every step of the mechanism is described here, and every figure in the paper is produced by a script that can be re-run.

The Monad is organised around one operator, the Null Operator, written ⌀_RB and typed 0_RB. This is the core functionality of the Monad's Speaking Engine. The prompt is fed directly to σ_RB, the sixteen-dimensional input of 0_RB, and the equations are run, and they output a sedenion pathway, which we are building with and around, and providing toolsets to use, in this attempt at teaching a single equation of mathematics how to speak English. At runtime, every variable begins as NULL rather than zero, so that the maths, and not the programmer, chooses the fit. The Null Operator was a happy little accident. While discussing other maths for other projects, and after amassing a collection of code toolsets and operators in the scope of an AI assistant instructed to watch for emergent operators (such as how an angle in polar coordinates indicates a vector direction from the origin), the assistant reported a ground-state object that those operators defined: a Hamiltonian "just sitting here", in the words of Claude Sonnet 4.6. That model supplied the original H_hat_RB, which, for the purposes of the Monad, became the Null Operator, 0_RB.

The architecture is a Wankel rotary engine with a sedenion at the centre of the rotor, so that the equation can watch itself, its input and its output from inside while it runs. This is the model used here; why it was chosen is left for later.

The Monad exists as two pieces of working code: a compiled monad, `ptol` (`PtolC/ptol.c`), and the Python monad it was built from, `scaled_mind_eye_boxkite_kernel_monad.py`. The scaled monad runs live and continuously, not as one pass from prompt to response, and it is designed to include chaotic, non-deterministic behaviour: as impurities doped into silicon create a transistor, the impurities are what create the function. Section 2 fixes the notation, disambiguations and conventions used throughout; Section 3 defines the machine in the order the code runs, beginning with the Null Operator; the later sections employ it, show it running, and record where it falls short.

---

2. Notation, disambiguations, and conventions followed in this paper

2.1 Notation

Every component described in this paper carries one label, applied consistently:

- **`ESTABLISHED`**: mathematics or computer science that predates this work, cited to its source.
- **`OURS`**: code or a design choice built for this work and not published elsewhere.
- **`THEORETICAL`**: designed, or partly attempted, but not reduced to running, verified code.
- **`OPEN`**: a question this paper cannot settle.
- The suffix **`:CALCULATED`** marks a result the paper's own code produced. It says a number came out of a run; it does not say a statement was proved.

Maths is written in Unicode, never LaTeX, and code identifiers are given as the code spells them. The symbols used:

symbol	code identifier	meaning
ψ	`v` (`ptol.c`), `psi` (the Python monads)	the sixteen-component element a prompt becomes
e_0 … e_{15}	index `k` = 0…15 of ψ	the sixteen basis elements; e_0 is the identity
⌀_RB, 0_RB	`HUB_PSI` = [1, 0, …, 0] (`scaled_mind_eye_boxkite_kernel_monad.py`)	the Null Operator; written ⌀_RB, typed 0_RB
σ_RB	`s_rb[k]` (`ptol.c`), `sigma_rb` (`rotary_rerun_monad.py`)	the sixteen-component vector σ_RB[k] = ψ[k]·ψ[k⊕4]
σ	`sig`, `PtolEye.sigma`	the dial: the power-law exponent in the projection
σ_self, Γ	`measure_sigma`, `measure_gamma`	the balance reading P_red / (P_red + P_blue), and Γ = 2σ_self − 1
Eye R, C, H, O, S	`TOWER_EYES`	five settings of the dial, σ = 1, ¾, ½, ¼, 0
J_red, J_blue, J_green	the red and blue channel groups	forward channels, return channels, and the interface between them (§2.2)
NULL	`None` / `NaN`, never 0	a variable with no value yet

2.2 Disambiguations

Citations. A citation with no tag is a source the code was built directly against: WordNet's data, the Infocom/Zork parser tradition, and de Marrais's box kite. A citation marked `Post-Hoc:` is a correspondence noticed after the code already worked. Nothing in this paper requires knowing either name to follow the code.

Three different σ. In this codebase "σ" names three things, each correct in its own file: **σ**, the dial of the projection (a power-law exponent, set by the Eye); **σ_self**, the balance reading derived from the sixteen components of ψ; and **σ_RB**, the product vector above. A fourth, σ_ζ (the coupling exponent in `ValaQuenta/modules/h_rb_hat`), does not appear in this paper.

Two red/blue assignments. Red and blue are the author's colours and are used as labels only. Which indices carry them is a code fact, and the two programs differ: in `ptol.c` the red (forward, cosine) channels are k ∈ {0–3, 8–11} and the blue (return, sine) channels are k ∈ {4–7, 12–15}; in `rotary_rerun_monad.py` and the scaled kernel the split is at bit 3, red k ≥ 8 and blue k < 8. Every result below says which program it was measured on.

`ptol` and `ptolemy`. `ptol` is the compiled monad built from `ptol.c` (with `monad.c`, `monad_harness.c`, `tokenizer.c`, `filter.c`). `ptolemy`, built from `main.c`, is the daemon, REPL and ingest program. This paper is about `ptol`; `ptolemy` appears only where stated.

Monad, monad. *The Monad* is the engine. *A monad* is one file that implements it (`monad.c`, `*_monad.py`).

Eye. An *Eye* in `TOWER_EYES` is one setting of the dial. The *Mind's Eye* of the Python monads is the class that reads a snapshot of ψ, and *Paper's Hands* is the class that emits.

NULL, 0, ∅. NULL is a variable with no value yet. 0 is a computed value. ∅_RB denotes the empty set, never the numeral zero; it is typed 0_RB for keyboard availability.

2.3 Conventions

- One running example, the prompt `what is a tree`, carries every named-step description; six test prompts are used for every measurement.
- Mechanism is given as named steps, tables and literal transcripts. Source is quoted verbatim only for the GPL v3 parts (the harness and everything built around it); none of the Monad's source is reproduced in this text. Companion notebooks (§19) run the real code.
- Every figure comes from a script that can be re-run (`VAPMIP/paper/make_paper.py`); measured times are marked as such.
- Where the code disagrees with its own comments or with a second program, the paper reports the disagreement as a result.

2.4 C versus Python

`ptol` is the compiled, live monad. `scaled_mind_eye_boxkite_kernel_monad.py` is the Python monad it was built from; Python is also where the code is readable. They are not line-for-line twins, and a result measured on one is not claimed for the other. The compiled monad is not self-contained: its word step and its sentence step each call Python (§6, §8).

2.5 The machine measured on

One ThinkPad X1 Carbon (6th generation, 20KH002XUS), Intel Core i7-8550U, no GPU, no network, Linux 6.8.

2.6 The objects this paper uses, defined plainly

- **Synset.** WordNet's basic unit: a set of words interchangeable in one meaning. `{car, automobile}` is one synset.
- **Relation pointer.** One of the 19 typed links between synsets (hypernym, hyponym, meronym, and so on); a word's WordNet entry in the store is the 19 counts.
- **Sedenion.** A 16-dimensional number built by repeated doubling: reals, complex numbers, quaternions, octonions, sedenions. Sedenion multiplication has zero divisors.
- **Identity.** The element e_0: multiplying by it changes nothing.
- **Zero divisor.** A non-zero number that multiplies another non-zero number to exactly zero.
- **Box kite, strut, assessor.** The name (de Marrais, 2000) for how the sedenion zero divisors arrange themselves: seven interlocking octahedra (one per strut) sharing 42 zero-divisor pairs (the assessors). Established mathematics this work is built against, not derived here.

2.7 Install and run

```
cd VAPMIP/PtolC && make                          # builds ptolemy and ptol; needs gcc, make, libm
python3 -m venv .venv && . .venv/bin/activate && pip install nltk
python3 -c "import nltk; nltk.download('wordnet')"
HOME=$(mktemp -d) ./ptol -r "what is a tree"     # sixteen numbers; ptol writes files into $HOME/.ptolemy in its
./ptol -say "what is a tree"                     # a sentence; run with your real HOME (§8)
python3 scaled_mind_eye_boxkite_kernel_monad.py  # the hub report and seven fiber throws
```

Part I: the machine, in the order the code defines it

3. The Null Operator

Code: `ValaQuenta/modules/box_kite/maths.py` (`multiply`, `e0_is_outside`, `fixed_point_gluing`, `fixed_point_weight`, `box_kites`, `chart_spectrum`); `VAPMIP/scaled_mind_eye_boxkite_kernel_monad.py` (`HUB_PSI`, `MindEyeBoxKiteKernel`).
Notebook: `VAPMIP/notebooks/paper/17_null_operator_and_hub.ipynb` (13 PASS, 0 FAIL).

3.1 Definition

In the code the Null Operator is one vector: `HUB_PSI = [1.0] + [0.0] * 15` (`VAPMIP/scaled_mind_eye_boxkite_kernel_m` the sixteen-component element with all its weight on e_0. e_0 is the identity of the sedenion product `multiply` (`ValaQuenta/modules/box_kite/maths.py:179–201`): multiplying any sixteen-vector by it on either side returns the vector, it preserves the norm, and it commutes with every basis element (300 random vectors; largest deviation 0). `ESTABLISHED` structure, `OURS:CALCULATED` checks.

3.2 Outside every chart

`e0_is_outside` returns three flags and one verdict. e_0 is not a point of PG(3,2) (the 15-point geometry of the sedenion's imaginary units); it is in none of the 42 assessors; and its associator vanishes against every pair of basis elements. The 42 assessors form seven box kites of six assessors each, no assessor in two kites, and each kite's graph is an octahedron (Laplacian spectrum {0, 4, 4, 4, 6, 6}). `OURS:CALCULATED`, except the kite structure, `ESTABLISHED` (de Marrais, 2000).

3.3 The hub the seven charts share

Two structures live on this one object. In the *products* the seven charts are disconnected: there are no edges between kites, and the glued atlas has seven zero modes, one per chart. In the *skeleton* (which basis indices appear in which chart) they overlap almost entirely: fourteen indices each lie in six of the seven charts, and exactly two lie in none, e_0 and e_8. `fixed_point_gluing` reports that the seven zero modes are seven copies of one constant and identify at e_0: the one point the charts share. `fixed_point_weight(v)` is $v[0]^2 / |v|^2$, the share of a vector's energy on e_0; it is exactly 1 for `HUB_PSI`.

3.4 NULL start

Every variable in the machine begins as NULL, not zero: a zero is a value, and a value selects a scale. §14 records what each program does with an empty or all-zero input.

---

4. The store

Code: VAPMIP/PtolC/monad3c.h; VAPMIP/PtolC/ptol.c (monad3_open: VAPMIP/PtolC/ptol.c:113–130, monad3_lookup: VAPMIP/PtolC/ptol.c:136–168); VAPMIP/PtolC/monad.c (monad_word_coords, monad_wm_get, monad_wm_set); VAPMIP/PtolC/ptolemy.h. **Notebook:** VAPMIP/notebooks/c/08c_ptol_store.ipynb (3 PASS, 0 FAIL).

4.1 One file, four fields of language

The four fields of language, words, context, semantics and phonetics, live in one memory-mapped file, PtolC/monad3_c.bin. Its layout is monad3c.h: a header; a word blob; one 16-byte WordRec per word (name offset and three indices); the learned (English) store, with per-word energy, age, firing and stratum arrays and a sparse co-occurrence matrix; the WordNet store, one 82-byte entry per word (name, part of speech, an offset, 19 signed counts, a depth weight); and the phonetic store. The file read for this paper: 48,209,731 bytes, 347,119 words, 164,283 learned entries, 146,743 WordNet entries, 123,455 phonetic entries, 2,248,064 co-occurrence edges. The resident service folds conversation into this file by design, so its size and counts change; the figures are those at the time of this run.

4.2 Opening and lookup

monad3_open maps the file read-only, checks the 8-byte magic MONAD3C, and keeps the base pointer; if the file is absent the program carries on without it. monad3_lookup(word, out[3], &degree) lowercases the word and runs one binary search over the WordRec table; on a hit it fills the three store indices (−1 where a store has no entry) and the word's co-occurrence out-degree. No parsing, no copy, no allocation. Lookups on this store (notebook 08c):

word	found	learned idx	WordNet idx	phonetic idx	co-occurrence degree	Gamma-Radial
tree	yes	563	134483	112979	292	-0.904887
what	yes	228	-1	119807	2697	—
piano	yes	21590	99564	84465	20	+nan
xyzzyq	no	-1	-1	-1	0	—

The nan for piano was chased to its source (ptol -M piano, OURS:CALCULATED): the store's WordNet entry for piano (pos 4) holds an all-zero relation vector, and monad3_gamma_radial returns NAN for log_code ≤ 0 by design, with the caller checking isnan. It is the data and the guard working, not a fault; the store's piano entry is the one with no relation pointers.

4.3 Word to slot and energy

A word's slot and energy come from its spelling alone. monad_word_coords (VAPMIP/PtolC/monad.c:170–220): (1) read the word as a base-95 number over the printable characters; (2) multiply by the 64-bit golden-ratio constant (Fibonacci hashing); (3) take the result as a fraction seed in [0, 1); (4) the slot is floor(seed × N) with N = 25,000; (5) the energy is E = MONAD_D_STAR + seed × (MONAD_OMEGA_ZS − MONAD_D_STAR), so every energy lies between the two constants of ptolemy.h, 0.24600 and 0.56714. The step that makes the hash usable was found by measurement: an earlier version did the fractional part in floating point, which loses the low bits past 2^{53}, so 62,961 of 101,916 WordNet tokens landed on slot 0 and survival was 0.00 % for every word of eight or more characters; after the integer version the largest pile-up was 14 (χ^2 3,833,102 → 100.0). monad_wm_get / monad_wm_set keep the word → slot map (open addressing); a slot's VocabEntry has a present flag, set when a word has been assigned to it.

word	slot (of 25,000)	E
tree	3109	0.285940
what	16194	0.454032
piano	2639	0.279904
neutron	7936	0.347950
good	19600	0.497781
morning	16537	0.458432
reactor	18415	0.482558
discovered	23100	0.542734

---

5. The question: a prompt becomes ψ

Code: VAPMIP/PtolC/ptol.c (project: VAPMIP/PtolC/ptol.c:282–293; measure_sigma: VAPMIP/PtolC/ptol.c:311–322; measure_gamma: VAPMIP/PtolC/ptol.c:333–341; TOWER_EYES; the raw path of main). **Notebook:** VAPMIP/notebooks/c/07c_ptol_projection_and_balance.ipynb (1 PASS, 0 FAIL).

5.1 The projection

Named steps, running example what is a tree (14 bytes). OURS.

1. **Bytes.** The prompt is read as bytes $c_1 \ldots c_n$.
2. **Oscillator bank.** Sixteen periods P[k] are the first sixteen primes: 2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53 (P[16]).
3. **Dial.** A number σ (the Eye's sigma; default ½) sets a power-law weight i^(−σ) on position i.
4. **Project.** Component k is the sum over positions i of $c_i \cdot i^{(-\sigma)} \cdot w_k(2\pi \cdot i / P[k])$, where w_k is **cosine** for k in 0–3 and 8–11 (the red, forward channels) and **sine** for k in 4–7 and 12–15 (the blue, return channels).
5. **Normalise.** The sixteen numbers are divided by their Euclidean length. ptol -r prints them, one per line; the line eye: … goes to standard error.

5.2 The balance reading

measure_sigma returns P_red / (P_red + P_blue), where P_red is the sum of squares of the red components and P_blue of the blue; it returns 0.5 if the total is below 10^{-15} (§14). measure_gamma returns Γ = 2σ_self − 1 (clamped to ±0.999999999) and u = 2·atanh(Γ) = ln(P_red/P_blue). Γ = 0 is the balance point.

5.3 σ_RB

For each k, s_rb[k] = v[k] · v[k⊕4] (the partner of k is k⊕4: 0↔4, 1↔5, 2↔6, 3↔7, 8↔12, and so on). ptol -r prints these sixteen products as its third block.

5.4 The five Eyes

An Eye is a named dial setting. TOWER_EYES defines five, R, C, H, O, S, with σ = 1, ¾, ½, ¼, 0; each also carries a theta and an aperture field, which the code does not read: the projection uses only the Eye's sigma. The balance reading of the same question at each Eye (σ_self; ½ would be balance):

prompt	chars	R (σ=1)	C (¾)	H (½)	O (¼)	S (0)	Γ at H
finger pointer	14	0.4186	0.2819	0.1501	0.0675	0.0490	-0.6998
what is a tree	14	0.4890	0.3570	0.2100	0.0962	0.0529	-0.5800
neutron generation in different reactor types	45	0.4376	0.3314	0.2454	0.2424	0.4372	-0.5092
good morning	12	0.4836	0.3726	0.2531	0.1534	0.0951	-0.4938
can you play the piano?	23	0.3373	0.1540	0.0307	0.0210	0.0836	-0.9385
the first reactor was built after neutrons were discovered	58	0.4438	0.2843	0.1304	0.0656	0.1218	-0.7392

5.5 Agreement with an independent restatement

A Python restatement of steps 1–5 (`VAPMIP/paper/evidence.py`, `project()`) agrees with `ptol -r` on all sixteen components of all six prompts to **4.9e-11** (the compiled engine prints ten decimals), and on σ_self at all five Eyes to **5.0e-11**. The C notebook extracts `project`, `measure_sigma` and `measure_gamma` verbatim from `ptol.c`, runs them, and compares with the real binary: max |extracted − real| = 4.87e-11. Over 2,000 random prompts at three dial settings P_red + P_blue = 1 to 4.4e-16 and P_red · P_blue never exceeded 0.250000 (the bound is ¼, reached at balance). `OURS:CALCULATED`.

5.6 What the source comments say that the running code does not do

Kept as results. (1) The `ptol.c` header says *"J_red × J_blue = d* = 0.24600 is conserved at ALL σ"*. No such quantity is constant: P_red · P_blue over the six prompts at five dial settings ranges from 0.021 to 0.250 (a value within 10^{-4} of 0.2460 occurs by chance at a rate of 0.31% per sample). (2) The comment on `measure_sigma` says σ_self tends to 1 at σ = 1 and to 0 at σ = 0. Over 2,000 random prompts σ_self at σ = 1 has median 0.410 (maximum 0.871), and at σ = 0 median 0.153 (minimum 0.026). (3) σ_self follows the dial only for short inputs. Fraction of random inputs for which σ_self rises monotonically as σ goes from 0 to 1, by input length:

input length (characters)	σ_self rises with the dial	fraction
0–9	118 of 118	100.0%
10–19	82 of 163	50.3%
20–29	0 of 181	0.0%
30–39	0 of 170	0.0%
40–49	0 of 169	0.0%
50–59	0 of 147	0.0%
60–69	0 of 154	0.0%

input length (characters)	σ_self rises with the dial	fraction
70–79	1 of 199	0.5%
80–89	0 of 213	0.0%
90 and over	0 of 486	0.0%

For inputs of 20 characters or more σ_self is U-shaped in σ: of 300 such inputs 300 rise monotonically from σ = ½ to σ = 1; the minimum sits at median σ = 0.25 (10th to 90th percentile 0.15 to 0.35); random lowercase strings of 40 letters give the same shape (median minimum at σ = 0.35), so it is a property of the projection on long inputs, not of language. `OURS:CALCULATED`.

---

6. Words from ψ

Code: `VAPMIP/PtolC/ptol.c` (`get_monad_words`: `VAPMIP/PtolC/ptol.c:398–442`; `monad3_word_gamma_radial`; `gr_log_anchor`); `VAPMIP/PtolC/ptol_layer.py` (`select_layer`, `word_at`, `assemble_path`, `respond`); `VAPMIP/wordnet_boxkite.py` (`context_addr`, `gamma_radial`). **Notebook:** `VAPMIP/notebooks/c/08c_ptol_store.ipynb`.

6.1 The word shadow

`get_monad_words` writes ψ and the primes of the firing components to a temporary file and runs `python3 ptol_layer.py --stdin --words-only` through `popen`. In `ptol_layer.py`: `_scan_layers` finds every `holcus_monad_*.bin` vocabulary in its bin directory and names a layer after each file; `select_layer` adds three scores per layer (from the prompt text, from the path of the sixteen scalars, and from σ_RB) and takes the highest, with English as the floor; `word_at(data, v)` maps a scalar v in [−1, 1] to an index in the layer's word table and searches outward for the nearest non-empty entry; `assemble_path` orders components by ascending |scalar|, the spiral from the zero divisor outward, and keeps each word once. The result is a set of words, one per firing component: the **shadow** of the geometry. For the six prompts the shadow words are unrelated to the prompt's own words (§8, §15).

6.2 The 19-count address and the Gamma-Radial reading

A word's WordNet entry is 19 counts, one per relation type. `gr_log_anchor` and `monad3_gamma_radial` (`VAPMIP/PtolC/ptol.c:206–213`) turn the 19 counts into one number: with a prime assigned to each relation (73, 79, 83, … 167), log_code = Σ count_i · ln(prime_i), and the reading is tanh(½ · ln(log_code / anchor)), where the anchor is Σ ln(prime_i) over all relations except hyponyms; it is NaN for an all-zero vector. `OURS`; the Python original is `wordnet_boxkite.gamma_radial`.

6.3 A recorded value that no longer reproduces

The comment above `monad3_gamma_radial` records that on 2026-09-21 the reading for "tree" was −0.151155, matching the paper-recorded value. Today the compiled monad's reading for "tree" is -0.904887, because the store's single WordNet entry for the word is now a verb sense. In Python the reading for each sense of "tree" is: `tree.n.01` -0.151155; `tree.n.02` -0.734935; `tree.n.03` -0.815375; `corner.v.02` -0.904887; `tree.v.02` -0.904887; `tree.v.03` -0.904887; `tree.v.04` -0.904887. The number −0.151155 belongs to the first noun sense (`tree.n.01`); which sense the store keeps for a word depends on how the store was built. `OURS:CALCULATED`; reported, not changed.

---

7. Learning and speaking in the C monad

Code: VAPMIP/PtolC/monad.c (monad_learn_ex: VAPMIP/PtolC/monad.c:450–581; monad_a_add, monad_a_get; monad_speak: VAPMIP/PtolC/monad.c:772–1047; monad_hear_fermat; monad_self_flush); constants in VAPMIP/PtolC/ptolemy.h. **Notebook:** VAPMIP/notebooks/c/09c_ptol_learning_step.ipynb (C kernel; extracts the β update, the prose_seen ladder and the co-occurrence coupling verbatim from monad.c and runs them).

7.1 Learning

monad_learn_ex(m, text, verbose, filetype) refuses PEM key or certificate material outright. For each sentence (split at ., !, ?, newline) it: (1) tokenises and keeps tokens token_accept allows; (2) for each token gets the slot and energy (§4.3) and raises the slot's β by E^2 × MONAD_ALPHA_LEARN × |sin(zeros[idx])| × π/2, capped at MONAD_BETA_SAT (7.552); (3) writes the word, its energy and present = 1 into the slot's VocabEntry if the slot was empty or the new energy is higher; (4) updates prose_seen (0 unseen, 1 prose only, 2 WordNet, 3 both); (5) for every pair of accepted tokens within MONAD_WINDOW_MAX (12) positions adds a co-occurrence edge w = E_i · E_j · couple / ((|Δγ| + MONAD_GAP) × d), with couple 2 if the two slots have the same pole and 1 otherwise, d the distance in tokens and Δγ the difference of the slots' positions. The prose_seen ladder is order-dependent: WordNet then prose reaches 3, prose then WordNet does not (found by running the extracted arithmetic in notebook 06.5).

7.2 Speaking

monad_speak(m, query, max_tokens, verbose): (1) seed the field J from the query's slots; (2) spread J to neighbouring slots with a decay of MONAD_SPREAD_DECAY over MONAD_SPREAD_RADIUS; (3) two propagation passes over the co-occurrence edges, both reading the field before either pass (the forward pass and a reverse pass), so words raised by both are the nodes; (4) the field sets its own word count from the balance of its two energy sums; (5) find the pointer slot nearest the median position of the query's slots, and update affect (irritation grows if the pointer is in the same region as last time); (6) walk the slots by a golden-angle step of round(N/φ^2), emitting words whose gate is positive; (7) choose the first word by closeness of its energy to the field's centroid; (8) append the reply to the self-referential buffer, which monad_self_flush later feeds back to monad_learn. monad_hear_fermat reads a spaced-character code in incoming text and injects a proportional β boost; variants monad_speak_wick, _oct and _charge project the same field on different faces. OURS.

---

8. The sentence path of the compiled monad

Code: VAPMIP/PtolC/ptol.c (console_speak: VAPMIP/PtolC/ptol.c:984–1084; infer_speech_act; get_monad_sentence); VAPMIP/engine/grammar/ptolc_bridge.py; VAPMIP/engine/grammar/generate.py (ParseSpec, build_response). **Notebook:** VAPMIP/notebooks/paper/18_ptol_sentence_bridge.ipynb (4 PASS, 1 FAIL).

8.1 What C hands over

console_speak(prompt): (1) project the prompt at Eye H and normalise (§5); (2) threshold at peak / φ; (3) order the components by ascending magnitude; (4) get_monad_words returns one word cell per firing component (§6.1); (5) the distinct words, in spiral order, are the **word bag**; (6) compute σ_self and Γ; (7) if there are words, infer_speech_act reads the prompt: why, how, where, who_Xs, define_X (a "what" without "do"), what_does_X_do, or yesno if the first word is an auxiliary, and defaults to what_does_X_do; (8) get_monad_sentence writes three lines to a temporary file (GAMMA,

ACT, WORDS) and runs python3 -m engine.grammar.ptolc_bridge from the repository root; (9) if the bridge exits zero with a non-empty line, that line replaces the word bag; otherwise the word bag stands. The guaranteed fallback is the word bag.

8.2 The bridge

ptolc_bridge.py reads the three lines, builds a ParseSpec (speech act, verb, topic lemmas) and calls build_response in engine/grammar/generate.py, which fills a VerbNet-based sentence frame with WordNet words and gates the fill. On success it prints one line.

8.3 What it returns

ptol -say must run with the real HOME: it writes no files, but the bridge needs the user's site-packages and ~/nltk_data; with a scratch HOME the bridge cannot run and ptol silently returns the word bag, about ten times faster. (A scratch-HOME run mistaken for the real path was found and corrected while this paper was being written.) The compiled monad on the six prompts (plain ptol with a scratch HOME, -say with the real one, and one line of the compiled REPL):

prompt	ptol (plain)	ptol -say (real HOME)	compiled REPL line	parser line
finger pointer	gift-gems tropheus · spurned · · ox-hide · · · converteret *trolley *wittgenstein *rapa *nuys *mais	Wittgenstein trolleys wittgenstein.	changelogs construction loop kinase	I don't know 'finger'. Try: look, take, put, go, use, open, say, find, make, ask
what is a tree	horseback whirleth · tilt · · innovators marsyas arrow-swift catos philippopolis *polysyllables *ise *· *· *joost	Time is able.	kite its engine probe	0/identity "identity is tree"
neutron generation in different reactor types	weaver-of-peace glory-decked monenda · hard-scrabble massy · *alwayes *squirting *yom *· *· *convinto *· *· *·	Priest relates country.	kite its real gawo	I don't know 'neutron'. Try: look, take, put, go, use, open, say, find, make, ask
good morning	· · · penned · colourless aliment gaule · *seemes *gurls *aughty *winningly *birdseye *· *lexicogrammatical-word	Priest relates country.	libx alternator cdotImodinet	I don't know 'good'. Try: look, take, put, go, use, open, say, find, make, ask

prompt	`ptol` (plain)	`ptol -say` (real HOME)	compiled REPL line	parser line
can you play the piano?	`water's-edge folcwalda's refute industriously assiduously · · confabulations patrolling swarming-time vagabonds alemannicus *marja-liisa *· *mcarthur *· · · commissioning ebb-tide phiz · unappropriated · · · enoch lukewarm *countryfolk *collectives *nodes *·`	`Time is able.`	`kite open and where`	`I don't know 'can'. Try: look, take, put, go, use, open, say, find, make, ask`
the first reactor was built after neutrons were discovered		`George relates country.`	`kite pores helicities down`	`I don't know 'the'. Try: look, take, put, go, use, open, say, find, make, ask`

For these prompts the compiled engine's words are not related to the prompt's own words, and two different prompts can return the same sentence. With an empty word list the bridge still returns a sentence (`Time is able.`), although its docstring says a bad word list prints nothing and exits non-zero (notebook 18). `OURS:CALCULATED`.

---

9. The parser and the harness

Code: `VAPMIP/PtolC/zork.c` (`VERBS[]`, `zork_parse`: `VAPMIP/PtolC/zork.c:201–344`), `VAPMIP/zork_parser.py`; `VAPMIP/PtolC/monad_harness.c`, `monad_harness.h`; `VAPMIP/PtolC/ptol.c` (`run_console`). **Notebook:** `VAPMIP/notebooks/paper/19_parser_tables.ipynb` (4 PASS, 2 FAIL).

9.1 The parser

Verb-governed, in the Infocom tradition (Lebling, Blank and Anderson, 1979). `zork_parse(input)`: (1) a leading `/` is a slash command and bypasses the monad; (2) tokenise; (3) direction shorthand; (4) look the first token up in the verb table, with a six-letter prefix expansion (the Infocom rule); the verb selects one of **sixteen operators** (identity, negate, bind, name, apply, abstract, branch, iterate, recurse, allocate, query, dereference, compose, parallelize, interrupt, emit); (5) collect the remaining tokens as nouns, skipping articles and resolving *it* to the last noun; (6) split at a preposition into first and second noun; (7) build the prompt handed to the monad. Example: `what is a tree` parses to operator e0/identity with the prompt `identity is tree`. The compiled and Python tables, compared exhaustively (`OURS:CALCULATED`):

- Verbs in the compiled table: **197**; in the Python table: **207**. Operators covered: 16 of 16 (compiled), 16 of 16 (Python).
- In the Python table and not the compiled table: **10** (`again`, `and also`, `break open`, `g`, `look for`, `these`, `those`, `type`, `with`, `z`); the other way: **0**.

- Shared verbs that map to **different operators: 5**: `break` (compiled 1, Python 14); `compose` (compiled 9, Python 12); `print` (compiled 12, Python 15); `scan` (compiled 0, Python 7); `write` (compiled 9, Python 12).
- Cause: the Python table lists six verbs under two operators each and keeps the last; the compiled table lists each verb once. Five of the six then differ (`ask` agrees). The header of `zork.h` says the table "mirrors zork_parser.py exactly".

Verbs per operator in the compiled table: identity 14, negate 14, bind 15, name 13, apply 14, abstract 13, branch 15, iterate 11, recurse 11, allocate 13, query 13, dereference 11, compose 9, parallelize 9, interrupt 11, emit 11.

9.2 The harness

`monad_harness.c` is the seam `ptol` crosses to drive the Python tabbed curses console: `ptol -w` forks the console onto the terminal and keeps the other end of a socket pair, reading newline-delimited JSON frames (`attach`, `say`, `cmd`, `mode`, `ping`, `quit`) and answering with `chat` frames. A passive "radio" frame carrying a support line is parsed (`mh_parse_support_line`) and folded into the Monad (`mh_ingest_support`). Frames are GPL v3 harness code.

Part II: the Python monad the compiled one was built from

10. The geometry

Code: `VAPMIP/rotary_rerun_monad.py` (`BoxKite`: `VAPMIP/rotary_rerun_monad.py:325–493`; `MindsEye`: `VAPMIP/rotary_rerun_monad.py:565–663`; `PapersHands`: `VAPMIP/rotary_rerun_monad.py:670–709`); `ValaQuenta/modules/box_ki`
Notebooks: `VAPMIP/notebooks/tuning_the_engine/phase_25.ipynb`, `phase_39.ipynb`.

`BoxKite` is built once from the box-kite mathematics of §3: it holds the seven kites, the struts, the 42 assessors, a monotonic `signature` (a counter, not an object id), and a `descent` hash of its two parents. `_Bound` is the base of the two faces: a subsystem works alone before any shared language exists, and binding it to a `BoxKite` is checked, not assumed; a reference across signatures is refused. `MindsEye.snapshot(ψ)` reads one instant across all dimensions and never emits: it returns `p_red`, `p_blue` (red = components $k \geq 8$, blue = $k < 8$, §2.2), `sigma_self`, `R` and `e` (the square roots), `trochoid_loss` = $|R - e|$, σ_RB, and the lit struts. `PapersHands` writes: `emit_pathway(strut)` puts a strut's lineage in build order; `relate(eye, ψ)` reads a snapshot and writes it as ordered relations. Reading and writing combined is speech, which is why both must bind the same box kite.

11. The dial

Code: `VAPMIP/19D_boxkite_context_monad.py` (`GaugeEye.continuous_snapshot`: `VAPMIP/19D_boxkite_context_monad.py:100–`
`GaugeHands.emit_steered`: `VAPMIP/19D_boxkite_context_monad.py:148–199`). **Notebook:** `VAPMIP/notebooks/tuning_the_eng`

`GaugeEye` extends `MindsEye` by one operation. `continuous_snapshot(ψ)` returns everything the snapshot returns plus, read at the same instant in the same call, the **discrete** reading (`is_zero_divisor`, `nearest_assessor`, from `chart_of`) and the **continuous** reading (`a_matrix_basin_windspeed`, from `local_curvature`). `GaugeHands.emit_steered(s0, target_rho, max_steps)` is the continuous companion on the emitting side: a field-steered walk from a starting complex point, raising `GaugeSteeringFailed` if it cannot reach its target. The pair of readings, discrete and continuous, is the dial.

12. The kernel: hub, fiber, run

Code: VAPMIP/scaled_mind_eye_boxkite_kernel_monad.py (MindEyeBoxKiteKernel, FiberThrow: VAPMIP/scaled_mind_eye_box
Notebook: VAPMIP/notebooks/paper/20_scaled_kernel_fibers.ipynb (6 PASS, 0 FAIL).

MindEyeBoxKiteKernel composes one BoxKite, one GaugeEye and one GaugeHands. hub_report() confirms the hub is real before it is used (e0_is_outside, fixed_point_gluing, the hub's weight). throw_fiber(strut, depth, gamma_companion) is one excursion from the hub: (1) build ψ fresh from HUB_PSI: ψ[0] = √(1 − depth²) and ψ[a] = ψ[b] = depth/√2 for the first assessor (a, b) of the strut's kite; (2) read continuous_snapshot(ψ); (3) read fixed_point_weight(ψ); (4) optionally run emit_steered from the complex point (strut/7, 0.15); (5) return a FiberThrow: strut, depth, assessor, ψ, the fixed-point weight, the discrete and continuous readings, σ_self, and the optional companion step. Because every throw rebuilds ψ from the hub, "return to the hub between fibers" is a property of the construction. run_boxkite(plan) is one throw per (strut, depth) pair. The seven fibers at depth 0.6:

strut	assessor	fixed_point_weight	is_zero_divisor	windspeed	σ_self (kernel split)
1	(2, 3)	0.640000	False	+0.0000	0.0000
2	(1, 3)	0.640000	False	+0.0000	0.0000
3	(1, 2)	0.640000	False	+0.0000	0.0000
4	(1, 5)	0.640000	False	+12.0000	0.0000
5	(1, 4)	0.640000	False	+12.0000	0.0000
6	(1, 7)	0.640000	False	+12.0000	0.0000
7	(1, 6)	0.640000	False	+12.0000	0.0000

The fixed-point weight at every landing is $1 - \text{depth}^2$ (0.64), the same number at every chart, to 10^{-16} over all seven struts and five depths. A throw leaves energy only on e_0 and the chosen assessor's two indices; throwing a strut again after other throws gives the identical landing; a depth outside [0, 1] is refused. OURS:CALCULATED.

13. The encounter: from a prompt to a sentence in the Python monad

Code: PtolemyDesktop/ptolemy_console.py (build_encounter_19d, ScaledMindEyeMonadAdapter.say); VAPMIP/19D_rotary_boxkite_monad.py (Monad3CStore, NineteenDMindsEye.create_context, RotaryBoxKite19D.process_input); VAPMIP/ptolemy_monad.py (infer_direction); VAPMIP/engine/grammar/generate.py. **Notebook:** VAPMIP/notebooks/paper/21_scaled_adapter_encounter.ipynb (6 PASS, 0 FAIL).

13.1 Null-first construction

build_encounter_19d(text, …) builds a parse specification only from what text resolves to, and otherwise returns an explicit non-sentence result: (1) split the text into words; the **verb** is the first word with a WordNet verb sense; the **leaves** are the other words with any WordNet sense; (2) no verb, or no leaves: return 19D: nothing resolved … NULL, not fabricated; (3) create_context(leaves) reads each leaf's current 19-count vector from monad3_c.bin (Monad3CStore) or, if the store lacks it, from WordNet directly, and records which was which; (4) infer_direction(root_vector) picks the dominant non-branching relation of the summed root and maps it to a direction: classify, enumerate, decompose, situate, characterize, explain, imply, associate, compare, contextualize, localize or register; if the root is all-zero the direction is observe and the result is reported as uncommitted; (5) a direction with no speech act returns 19D: direction=… has no speech_act mapping yet − reporting the real finding rather than forcing one; the map covers classify, enumerate and characterize (define_X), decompose (how), situate (where), explain and imply (why); (6) otherwise a ParseSpec goes to RotaryBoxKite19D.process_input, which returns the surface sentence, a verdict and the licensed frames.

13.2 The hub fiber of the adapter

`ScaledMindEyeMonadAdapter.say` appends to the sentence its direction, verdict and licensed frames, and one hub fiber: `strut = ring_code % 7 + 1` and `throw_fiber(strut)` at the default depth 0.6. The ψ of that throw is built from the hub (§12), not from the prompt; the prompt chooses only the strut. The adapter's own docstring records that a throw per sub-ring is the next step and is not built.

13.3 What it returns

The six prompts, first round (cold: the first call of a process also reads WordNet) and second (warm), `PYTHONHASHSEED=0`:

prompt	kind	reply (warm, seed 0)
finger pointer	scaled_mind_eye	`Pointer fingers. [direction:decompose verdict:prime licensed:['SV', 'SVO']] [hub fiber: strut=6 fixed_point_weight=0.6400 is_zero_divisor=False]`
what is a tree	scaled_mind_eye	`A ises tree. [direction:situate verdict:prime licensed:['SVO']] [hub fiber: strut=5 fixed_point_weight=0.6400 is_zero_divisor=False]`
neutron generation in different reactor types	19D_unmapped	`(19D: direction='compare' has no speech_act mapping yet — reporting the real finding rather than forcing one: leaves=['neutron', 'generation', 'in', 'different', 'reactor'])`
good morning	19D_null	`(19D: nothing resolved — no WordNet verb found, 2 content word(s) resolved. NULL, not fabricated — no sentence generated.)`
can you play the piano?	scaled_mind_eye	`Play cans piano into piano. [direction:decompose verdict:prime licensed:['SV', 'SVO', 'SVOA']] [hub fiber: strut=5 fixed_point_weight=0.6400 is_zero_divisor=False]`
the first reactor was built after neutrons were discovered	19D_unmapped	`(19D: direction='compare' has no speech_act mapping yet — reporting the real finding rather than forcing one: leaves=['first', 'reactor', 'built', 'after', 'neutrons', 'were', 'discovered'])`

The second round returned exactly the replies of the first within the process. Across five fresh processes with hash seeds 0–4, none of the six prompts give more than one reply. Figure 1 is a screenshot of the curses console running an ancestor of this monad, the box-kite monad (`rotary_rerun_boxkite`, signature 1, per its status bar), on the original laptop (an HP EliteBook 820 G3) on 2026-09-11: each reply is a sentence frame chosen by the dominant relation of the prompt's context, with the direction in brackets and the words that fired.

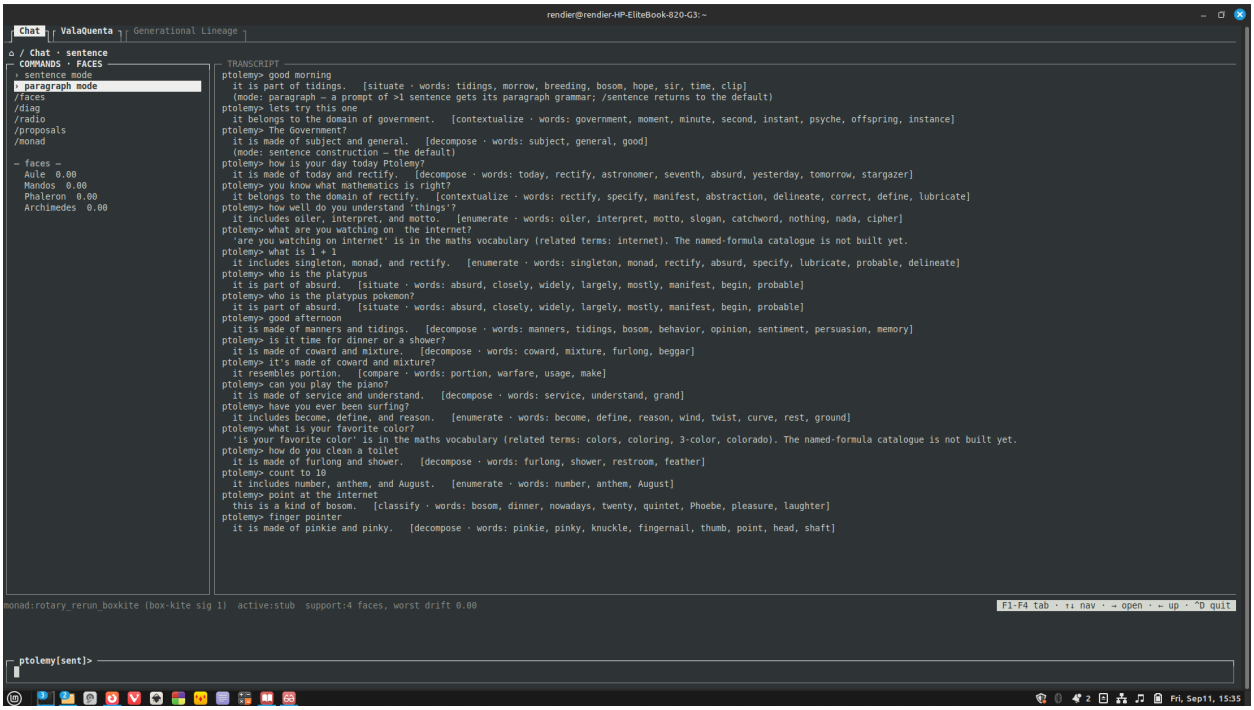


Figure 1: Figure 1. The tabbed curses console, Chat tab in sentence mode, running the box-kite monad (`rotary_rerun_boxkite`), 2026-09-11. Each reply carries its direction in brackets (situate, contextualize, decompose, enumerate, classify, compare) and the words that fired. Replies of the form "is in the maths vocabulary … The named-formula catalogue is not built yet" are the console reporting what it does not have. File: `VAPMIP/paper/figures/console_rotary_rerun_boxkite_2026-09-11.png`.

14. Fault handling and NULL behaviour

Code: `VAPMIP/rotary_rerun_monad.py` (`Fault`, `Status`, `Relation`, `Ledger`, `Harness`: `VAPMIP/rotary_rerun_monad.py:1639–3412` `ValaQuenta/modules/box_kite/maths.py` (`fixed_point_weight`); `VAPMIP/PtolC/ptol.c` (`measure_sigma`); `PtolemyDesktop/ptolemy_console.py` (`build_encounter_19d`). **Notebook:** `VAPMIP/notebooks/paper/22_fault_ledger_and_nu` (4 PASS, 0 FAIL).

14.1 Every fault is a record

`Harness.run(ψ, currents)` runs every relationship check and appends one `Relation` to the `Ledger` for each: a name, a claim, an expected and an observed value, a `Status` (`HOLDS`, `VIOLATED`, `DEGENERATE`, `UNTESTED`) and, for a violation, a `Fault`: **MATHS** (both sides were measured and disagree: the claim is false) or **CODE** (the check could not execute as written: the claim is unjudged). The ledger never filters or compacts, and a violated record is never left unclassified. On the deliberately unbalanced currents of the demonstration: 124 records; 122 HOLDS, 1 VIOLATED, 1 DEGENERATE, 0 UNTESTED; faults: 1 MATHS, 0 CODE; the one fault is `current.neutral` (expected 0.0, observed 0.024).

14.2 What each program returns for an all-zero input

The rule of §3.4 is that a variable with no value is NULL. What the code does with an all-zero input (a census; each entry is a data point):

- **keeps NULL:** `rotary_rerun_monad.MindsEye.snapshot(zeros)['sigma_self']` → nan
- **supplies a value:** `box_kite.fixed_point_weight(zeros)` → 0.0
- **supplies a value:** `ptol -r ""  (σ_self printed)` → 0.5000000000
- **supplies a value:** `ptol.c measure_sigma:`if (total < 1e-15) return 0.5;“ → returns 0.5 for a zero vector
- **keeps NULL:** `ScaledMindEyeMonadAdapter.say('good morning') (no verb resolves)` → (19D: nothing resolved — no WordNet verb found, 2 content word(s) reso

---

Part III: the two together

15. The two programs compared

All measured in this paper; each row says which program.

item	compiled `ptol`	Python monad	result
the projection (§5)	`ptol -r`	restatement	agree to 4.9e-11
red/blue channels (§2.2)	cosine/sine, k ∈ {0–3, 8–11} / {4–7, 12–15}	split at bit 3 (red k ≥ 8)	**differ**: σ_self is not one shared quantity
parser verb tables (§9.1)	197 verbs	207 verbs	5 verbs map to different operators; 10 verbs only in Python
header comments (§5.6)	two claims not reproduced	n/a	recorded
the recorded windspeed of “tree” (§6.3)	-0.904887 now	−0.151155 on `tree.n.01`	the store keeps a verb sense
sentence path (§8, §13)	word bag, or a sentence via Python (~12 s)	sentence, explicit non-sentence, or NULL	different mechanisms
what the prompt reaches (§13.2)	ψ drives the word shadow	the prompt chooses only the strut of the one fiber	the adapter does not pass the prompt’s ψ to the kernel
hash-seed dependence	none observed	none of six prompts vary	—

16. Running it

Time is measured fresh on every build of this paper and is not part of the `--check` comparison.

prompt	`ptol` ms	`ptol -say` s	`ptolemy -r -n` ms (whole process)	scaled adapter cold ms	scaled adapter warm ms
finger pointer	804	13.0	237	12117.6	5.58

prompt	`ptol` ms	`ptol -say` s	`ptolemy -r -n` ms (whole process)	scaled adapter cold ms	scaled adapter warm ms
what is a tree	788	11.3	228	295.1	43.38
neutron generation in different reactor types	798	11.4	226	11.0	0.27
good morning	792	11.3	241	3.0	0.08
can you play the piano?	809	11.2	228	447.4	148.91
the first reactor was built after neutrons were discovered	799	11.5	231	3.3	0.49

Adapter: construction 0.84 s; the first call of a process also reads WordNet (cold column). `ptol -r` (the sixteen numbers alone) takes about 2 ms. Compiled figures are the faster of two runs where two were taken (`-say` once, about 12 s) and include process start; the compiled REPL's per-line cost inside one session is the recorded 28.8 ms over 600 lines.

Energy is not re-measured by this paper's generator (it needs the laptop unplugged). It is taken from the recorded run of 2026-10-13/14 (`VAPMIP/benchmarks/all_voices/`, `ContextPlease/claude/scratchpad/2026-10-13_monad_energy/`): whole-laptop discharge on battery, display and operating system included, so an **upper bound** on the engine's own share. Mean power then: 9.255 W. A reply of t ms costs about 9.255 × t / 3.6 µWh. The compiled REPL measured 28.8 ms per line over 600 lines and 66.6 µWh per reply on that run; the engine's own share is below the method's resolution (idle drifted from 7.57 W to 10.35 W). A warm reply of the Python monad here is about 85.1 µWh; a sentence through the compiled monad's bridge, at the measured seconds, about 30 mWh. The public per-query figures are 0.24 Wh for a median Gemini text prompt (Vahdat et al., 2025), 0.34 Wh for an average ChatGPT query (Altman, 2025) and roughly 0.3 Wh (You, 2025). `OURS`, derived from recorded measurements; the public figures are `ESTABLISHED` only as stated by those sources.

17. Tuning the Engine: selected attempts, and the surface

The engine was tuned by trying a method, running it, and keeping what the run showed. The record is 45 pages (`VAPMIP/docs/wiki/Tuning-the-Engine/`) with one executed notebook per page (`VAPMIP/notebooks/tuning_the_engine/`, 44 notebooks: 933 PASS, 85 FAIL, 177 PROSE, 9 SKIP). Here are three of the most abstract methods, then the Navier-Stokes-revised ones, then the surface. Each is given as the method, the code, and what the notebook measured; a claim the notebooks do not reproduce is kept and labelled.

17.1 Definition from above (wiki 06; notebook `phase_06.ipynb`, 8 PASS, 0 FAIL, 4 PROSE)

The method. A layer cannot be defined from inside itself; the gaps of a layer (zero divisors, non-associativity, non-commutativity, non-ordering) are the shadow of the layer that defined it, so each layer is defined from the one above. *The code.* The repeated-doubling chain of §2.6 loses one property per step, and the engine reads a strut's build order: `BoxKite.lineage(strut)` (§10), `MindsEye.generations_present`. *Measured* (notebook): ℂ is commutative and ℍ is not; ℍ is

associative and 𝕆 is not; 𝕆 is alternative and 𝕊 is not; basis-unit zero divisors first appear at 𝕊, none in ℝ, ℂ, ℍ, 𝕆; the zero divisors of 𝕊 are the 84 vectors e_a ± e_b. The lineage of each strut, as `BoxKite.lineage` returns it:

strut	lineage
1	ranking → division
2	factors → division
3	ranking → factors → division
4	GROUPING → division
5	ranking → GROUPING → division
6	factors → GROUPING → division
7	ranking → factors → GROUPING → division

17.2 The fixed point as the one reference (wiki 09; notebook `phase_09.ipynb`, 7 PASS, 0 FAIL, 4 PROSE)

The method. Of the points of a symmetric dial only the fixed point of the symmetry s ↦ 1 − s can be named without reference to anything else; every reading is taken relative to it. *The code.* The default Eye is H (σ = ½) and Γ = 0 is balance (§5.2, §5.4). *Measured:* the notebook confirms that s = 1 − s has the unique real solution ½ and reproduces the checks the page lists; in the engine, no test prompt sits at balance, since Γ at Eye H lies between -0.94 and -0.49 for the six prompts.

17.3 The negative-space response (wiki 15; notebook `phase_15.ipynb`, 4 PASS, 2 FAIL, 3 PROSE)

The method. The prompt's quietest component, k_min = argmin |x[k]|, is a zero-divisor contact; the response lives on the other arm of that pair, k_pair, so that the algebra, not the engine, selects the response space. *The code.* The projection of §5 supplies x; the pairs are those of §3.2. *Measured, and kept:* the notebook finds that products of basis units never vanish (zero divisors are combinations e_a ± e_b, not basis units), and that a dimension does not have exactly one pair-mate dimension (the pair-mate of a zero divisor is a set of four). The statement that the algebra restricts the response to a proper subset of the fifteen imaginary dimensions does hold. The method as written does not; it stays in the record as a result.

17.4 The Navier-Stokes-revised methods (notebooks `10_revised_navier_stokes.ipynb`, `11_language_as_navier_stokes.ipynb`)

The method. The two notebooks argue that the Navier-Stokes equation lacks an imaginary unit (notebook 10, with an inviscid Burgers illustration of a shock as crossing characteristics), and that the speaking step of the monad plays the part of the revised equation (notebook 11), with the mapping:

Navier-Stokes term	`speak()` equivalent (notebook 11)
velocity field u	the field J (the activations of the words)
time derivative of u	the change of J between calls
non-linear commutator	the interaction of the red and blue currents
pressure gradient	the gradient of the β field
viscosity	the scoring of candidate words (`sigma_candidates`)
forcing f	the input text (`hear`)
spectral floor	`GAP = 0.000707`, the emission threshold

THEORETICAL: this paper tests none of the physical claims. What the code contains is the sequence of §7.2 (seed the field, spread it, propagate it in two passes, set the word count from the balance of the field's two energy sums, walk the slots, emit), and the constants of §4.3 and §7.

17.5 The surface: an artificial output layer

The method. A flow with no surface has nothing to read from; an artificial output layer, an interface between the continuous field and the discrete words, supplies one. *The code.* In the compiled monad the output layer is the word step of §6.1 (`word_at` and `assemble_path` in `PtolC/ptol_layer.py`: scalars in, words out) and the sentence step of §8; in the Python monad it is the sentence construction of §13. The diagnostic that measures the interface is `halocline_report` (`VAPMIP/monad.py:1803–1896`, method of `VAPMIP/monad.py`): it computes the two currents J_pos and J_neg from the window and the prompt, a σ per word as J_pos / (J_pos + J_neg) (½ below a relative threshold of 0.001 of the peak), the words closest to σ = ½ (the "SOFAR channel"), a surface tension of 1 − `noether_violation()` (not below 0), and a compressibility equal to the zero-divisor density of the window (`fermat_scan`). *Measured* (notebook `phase_02b.ipynb`, 6 PASS, 2 FAIL, 2 PROSE): the report returns every key the page promises; the surface tension equals 1 − the violation; a zero divisor annihilates only its specific partners (exact to 10^{-12}). Two checks fail and are kept: that the field operates on the halocline ($\bar{\sigma}$ = ½) and that the compressibility is non-zero somewhere.

18. Limitations and conclusion

1. The compiled monad's words are, for these six prompts, unrelated to the prompt's own words (§8.3); the words and sentences related to the prompt come from the Python monad, and only for 3 of six prompts.
2. The compiled monad needs Python for its word step and its sentence step; with a scratch `HOME` the sentence step silently falls back to the word bag.
3. Two statements in the compiled source's comments are not reproduced (§5.6); σ_self follows the dial only for short inputs.
4. σ_self is defined differently in the two programs (§2.2, §15).
5. The adapter passes the prompt only as a strut choice to the kernel; a throw per sub-ring is not built (§13.2).
6. The store keeps one WordNet entry per word, and for "tree" it is a verb sense (§6.3).
7. The parser tables disagree on 5 verbs (§9.1).
8. A zero input is handled four different ways across five paths; only two keep NULL (§14.2).
9. Context flow (carrying a prompt's topic through a whole reply) is not built.

The Monad as it stands turns a prompt into a sixteen-component element by a fixed projection and reads words from a store of WordNet's relations, with no gradient descent and no trained weights, on one laptop CPU. It does so with the limits above, and the code that does it is listed in §19.

References

Altman, S. (2025). The Gentle Singularity. `blog.samaltman.com`.

Bird, S., Klein, E., Loper, E. (2009). *Natural Language Processing with Python.* O'Reilly.

de Marrais, R. P. C. (2000). The 42 Assessors and the Box-Kites They Fly: Diagonal Zero-Divisors in the Sedenions. arXiv:math/0011260.

Fellbaum, C. (ed.) (1998). *WordNet: An Electronic Lexical Database.* MIT Press.

Lebling, P. D., Blank, M. S., Anderson, T. A. (1979). Zork: A Computerized Fantasy Simulation Game. *IEEE Computer* 12(4), 51–59.

Miller, G. A. (1995). WordNet: A Lexical Database for English. *Communications of the ACM* 38(11), 39–41.

Vahdat, A., et al. (2025). Measuring the environmental impact of AI inference. *Google Cloud Blog*, 21 August 2025.

You, J. (2025). How much energy does ChatGPT use? *Epoch AI, Gradient Updates*, 7 February 2025.

*Marked in the ledger `ContextPlease/claude/references/INDEX.md`: copies of Altman, You, Vahdat et al. and de Marrais were fetched and their title pages checked. Bird et al. (free online at `nltk.org/book`), Fellbaum, Lebling et al. and Miller were **not** fetched (a publisher bot-challenge stopped the attempt, or the work is a book); their details are from the author's records and have not been checked against a title page.*

---

19. ContextPlease inclusions and links

Every path is `REPO/directory/file` under `https://github.com/michaelrendier/`.

What ContextPlease is. ContextPlease (`ContextPlease`) is the author's versioned memory for work done with a language model: notes, protocols, skills, primers and scratch directories that let a fresh model session come up to speed with historical access instead of re-deriving what is already known. It exists because a model has no memory of its own between sessions.

19.1 Code, by section

- **§3 The Null Operator:** `ValaQuenta/modules/box_kite/maths.py`, `VAPMIP/scaled_mind_eye_boxkite_kernel_monad.py`
- **§4 The store:** `VAPMIP/PtolC/monad3c.h`, `VAPMIP/PtolC/ptolemy.h`, `VAPMIP/PtolC/ptol.c`, `VAPMIP/PtolC/monad.c`, `VAPMIP/PtolC/monad.h`, `VAPMIP/PtolC/monad3_c.bin`
- **§5 The question:** `VAPMIP/PtolC/ptol.c`
- **§6 Words from ψ:** `VAPMIP/PtolC/ptol.c`, `VAPMIP/PtolC/ptol_layer.py`, `VAPMIP/wordnet_boxkite.py`, `VAPMIP/PtolC/dump_boxkite_bin.c`
- **§7 Learning and speaking in the C monad:** `VAPMIP/PtolC/monad.c`, `VAPMIP/PtolC/ptolemy.h`
- **§8 The sentence path:** `VAPMIP/PtolC/ptol.c`, `VAPMIP/engine/grammar/ptolc_bridge.py`, `VAPMIP/engine/grammar/generate.py`
- **§9 The parser and the harness:** `VAPMIP/PtolC/zork.c`, `VAPMIP/PtolC/zork.h`, `VAPMIP/zork_parser.py`, `VAPMIP/PtolC/monad_harness.c`, `VAPMIP/PtolC/monad_harness.h`
- **§10 The geometry:** `VAPMIP/rotary_rerun_monad.py`, `ValaQuenta/modules/box_kite/maths.py`
- **§11 The dial:** `VAPMIP/19D_boxkite_context_monad.py`
- **§12 The kernel:** `VAPMIP/scaled_mind_eye_boxkite_kernel_monad.py`
- **§13 The encounter:** `PtolemyDesktop/ptolemy_console.py`, `VAPMIP/19D_rotary_boxkite_monad.py`, `VAPMIP/ptolemy_monad.py`
- **§14 Fault handling and NULL behaviour:** `VAPMIP/rotary_rerun_monad.py`
- **§17 Tuning and the surface:** `VAPMIP/monad.py`, `VAPMIP/docs/wiki/Tuning-the-Engine/00_index.md`, `VAPMIP/docs/wiki/Tuning-the-Engine/02b_the_halocline_j_blue_j_red_h_hat_rb.md`, `VAPMIP/docs/wiki/Tuning-the-Engine/06_definition_from_above_the_shadow_cascade.md`, `VAPMIP/docs/wiki/Tuning-the-Engine/09_the_void_named_itself.md`, `VAPMIP/docs/wiki/Tuning-the-Engine/15_the_negative_space_response_equati`
- **§2 licences and third-party notices:** `VAPMIP/NOTICE`, `VAPMIP/WORDNET_LICENSE.txt`, `VAPMIP/LICENSE`, `VAPMIP/LICENSE-GPL-3.0.txt`

19.2 Notebooks

- **§3:** VAPMIP/notebooks/paper/17_null_operator_and_hub.ipynb
- **§4:** VAPMIP/notebooks/c/08c_ptol_store.ipynb
- **§5:** VAPMIP/notebooks/c/07c_ptol_projection_and_balance.ipynb
- **§6:** VAPMIP/notebooks/c/08c_ptol_store.ipynb
- **§7:** VAPMIP/notebooks/c/09c_ptol_learning_step.ipynb
- **§8:** VAPMIP/notebooks/paper/18_ptol_sentence_bridge.ipynb
- **§9:** VAPMIP/notebooks/paper/19_parser_tables.ipynb
- **§10:** VAPMIP/notebooks/tuning_the_engine/phase_25.ipynb
- **§10:** VAPMIP/notebooks/tuning_the_engine/phase_39.ipynb
- **§11:** VAPMIP/notebooks/tuning_the_engine/phase_38.ipynb
- **§12:** VAPMIP/notebooks/paper/20_scaled_kernel_fibers.ipynb
- **§13:** VAPMIP/notebooks/paper/21_scaled_adapter_encounter.ipynb
- **§14:** VAPMIP/notebooks/paper/22_fault_ledger_and_null.ipynb
- **§16:** VAPMIP/notebooks/tuning_the_engine/phase_42.ipynb
- **§17:** VAPMIP/notebooks/tuning_the_engine/phase_06.ipynb
- **§17:** VAPMIP/notebooks/tuning_the_engine/phase_09.ipynb
- **§17:** VAPMIP/notebooks/tuning_the_engine/phase_15.ipynb
- **§17:** VAPMIP/notebooks/tuning_the_engine/phase_02b.ipynb
- **§17:** VAPMIP/notebooks/10_revised_navier_stokes.ipynb
- **§17:** VAPMIP/notebooks/11_language_as_navier_stokes.ipynb

19.3 Evidence, generator, skill and literature

- **the generator, its template and its figure:** VAPMIP/paper/evidence.py, VAPMIP/paper/make_paper.py, VAPMIP/paper/template.md, VAPMIP/paper/sec17.md, VAPMIP/paper/build_pdf.sh, VAPMIP/paper/figures/console_rotar 09-11.png
- **the notebook builders:** VAPMIP/notebooks/paper/pn.py, VAPMIP/notebooks/paper/build_paper_notebooks.py, VAPMIP/notebooks/paper/specs_paper.py, VAPMIP/notebooks/c/install_c_kernel.sh
- **Tuning the Engine, and its findings roll-up:** VAPMIP/notebooks/tuning_the_engine/FINDINGS.md, VAPMIP/docs/wiki/J_red-J_blue-0_RB-Record.md
- **the name chronology of the Null Operator:** Ainulindale/wiki/127_names_of_the_null_operator.md
- **scratch evidence (persistent scratchpad):** ContextPlease/claude/scratchpad/2026-10-15_vapmip_paper, ContextPlease/claude/scratchpad/2026-10-15_null_operator_names
- **the skill that decides how code is shown, and the introductions it was calibrated on:** ContextPlease/claude/skills/cs-paper-code-conventions/SKILL.md, ContextPlease/claude/skills/cs-paper-code-conventions/references/WhitePaperIntroExemplars
- **sources of the public energy figures (copies):** ContextPlease/claude/references/VAPMIPPaper, VAPMIP/paper/references
- **the ledger of downloaded literature:** ContextPlease/claude/references/INDEX.md

https://github.com/michaelrendier/